

Regular Expressions

A regular expression is a pattern consisting of a sequence of characters that matches the text.

Regular Expressions with SED

- In Unix, SED command stands for stream editor and it can perform lot's of function on file like, searching, find and replace, insertion or deletion.
- The most common use of SED command in UNIX is for substitution or for find and replace.
- By using SED you can edit files even without opening it, which is much quicker way to find and replace something in file, than first opening that file in VI Editor and then changing it.
- SED command in Unix supports regular expression which allows it perform complex pattern matching

Regular Expressions with SED

- Following is the general syntax for sed
 - `sed {options} /pattern/action file_name`
- Here, pattern is a regular expression, and action is one of the commands given in the following table.

Action	Description
P	Prints the line
d	Deletes the line
s/pattern1/pattern2/	Substitutes the first occurrence of pattern1 with pattern2

- If pattern is omitted, action is performed for every line as we have seen above.
- The slash character (/) that surrounds the pattern are required because they are used as delimiters.

SED Options

Option	Description	Example Usage
-n	Suppress automatic printing of pattern space.	sed -n '/pattern/p' file.txt
-e	Allows multiple commands or scripts to be executed.	sed -e 's/foo/bar/' -e 's/baz/qux/' file
-f	Read commands from a file instead of the command line.	sed -f script.sed file.txt
-i	Edit files in place (overwrite the original file).	sed -i 's/old/new/g' file.txt
-r	Use extended regular expressions in the script.	sed -r 's/(foo bar)/baz/' file.txt
-s	Treat multiple files separately rather than as a single continuous stream.	sed -s 's/foo/bar/' file1.txt file2.txt
-l	Specify the line length for the l command.	sed -l 72 'l' file.txt
--version	Display the version of sed installed.	sed --version

Deleting given Lines with sed

- In this example, the sed will delete the first line of file.txt and print the rest of the file. **\$ cat file.txt | sed '1d' | more**
- This deletes the first five lines. **\$ cat file.txt | sed '1, 5d' | more**

Range	Description
'4,10d'	Lines starting from the 4th till the 10th are deleted
'10,4d'	Only 10th line is deleted, because the sed does not work in reverse direction
'4,+5d'	This matches line 4 in the file, deletes that line, continues to delete the next five lines, and then ceases its deletion and prints the rest
'2,5!d'	This deletes everything except starting from 2nd till 5th line
'1~3d'	This deletes the first line, steps over the next three lines, and then deletes the fourth line. Sed continues to apply this pattern until the end of the file.
'4,10p'	Lines starting from 4th till 10th are printed

The Substitution Command

- The substitution command, denoted by `s`, will substitute any string that you specify with any other string that you specify.
- The following command substitutes the first occurrence on a line of the string `root` with the string `amrood`. **`$ cat /etc/passwd | sed 's/root/amrood/'`**
 - *amrood:x:0:0:root user:/root:/bin/sh*
daemon:x:1:1:daemon:/usr/sbin:/bin/sh
- It is very important to note that `sed` substitutes only the first occurrence on a line. If the string `root` occurs more than once on a line only the first match will be replaced.
- For `sed` to perform a global substitution, add the letter `g` to the end of the command **`$ cat /etc/passwd | sed 's/root/amrood/g'`**
 - *amrood:x:0:0:amrood user:/amrood:/bin/sh*
daemon:x:1:1:daemon:/usr/sbin:/bin/sh bin:x:2:2:bin:/bin:/bin/sh
sys:x:3:3:sys:/dev:/bin/sh

Sed selection command flags

- There are a number of other useful flags that can be passed in addition to the g flag, and you can specify more than one at a time.

Flag	Description
g	Replaces all matches, not just the first match
NUMBER	Replaces only NUMBERth match
w FILENAM	If substitution was made, then writes result to FILENAME
I or i	Matches in a case-insensitive manner

Use Regular Expression

- While matching patterns, you can use the regular expression which provides more flexibility. The following example matches all the lines starting with daemon and then deletes them **\$ cat testing | sed '/^daemon/d'**
- Following is the example which deletes all the lines ending with sh –
 - **\$ cat testing | sed '/sh\$/d'**
- The table lists four special characters that are very useful in regular expressions

Character	Description
^	Matches the beginning of lines
\$	Matches the end of lines
.	Matches any single character
*	Matches zero or more occurrences of the previous character

Matching Characters

- Some expressions that demonstrate the use of metacharacters.

Expression	Description
/a.c/	Matches lines that contain strings such as a+c, a-c, abc, match, and a3c
/a*c/	Matches the same strings along with strings such as aac, yaac, and aaaac
/[tT]he/	Matches the string The and the
/^.*\$/	Matches an entire line whatever it is
/ */	Matches one or more space
/^\$/	Matches blank lines

Matching Characters

- Following table shows some frequently used sets of characters

Expression	Description
[a-z]	Matches a single lowercase letter
[A-Z]	Matches a single uppercase letter
[a-zA-Z]	Matches a single letter
[0-9]	Matches a single number
[a-zA-Z0-9]	Matches a single letter or number

Character Class Keywords

- Some special keywords are commonly available to regexps. These are very useful for sed regular expressions as they simplify things and enhance readability.
- For example, the characters a through z and the characters A through Z, constitute one such class of characters that has the keyword[:alpha:]

Character Class	Description
[:alnum:]	Alphanumeric [a-z A-Z 0-9]
[:alpha:]	Alphabetic [a-z A-Z]
[:blank:]	Blank characters (spaces or tabs)
[:cntrl:]	Control characters
[:digit:]	Numbers [0-9]
[:graph:]	Any visible characters (excludes whitespace)
[:lower:]	Lowercase letters [a-z]
[:print:]	Printable characters (non-control characters)
[:punct:]	Punctuation characters
[:space:]	Whitespace

Ampersand Referencing

- The sed metacharacter & represents the contents of the pattern that was matched. For instance, say you have a file called phone.txt full of phone numbers, such as the following

```
5555551212
5555551213
5555551214
6665551215
6665551216
7775551217
```

- You want to make the area code (the first three digits) surrounded by parentheses for easier reading. To do this, you can use the ampersand replacement character

\$ sed -e 's/^[[[:digit:]][[[:digit:]][[[:digit:]]/(&)/g' phone.txt

```
(555)5551212
(555)5551213
(555)5551214
(666)5551215
(666)5551216
(777)5551217
```

Using Multiple sed Commands

- You can use multiple sed commands in a single sed command as follows– `$ sed -e 'command1' -e 'command2' ... -e 'commandN' files`
- Using the same mechanism, we can write the above phone number example as **`$ sed -e 's/^[[:digit:]]\{3\}/(&)/g' -e 's/)[[:digit:]]\{3\}/&-/g' phone.txt`**
(555)555-1213
(555)555-1214
(666)555-1215
(666)555-1216
(777)555-1217
- Note – In the above example, instead of repeating the character class keyword `[[:digit:]]` three times, we replaced it with `\{3\}`, which means the preceding regular expression is matched three times.

Back References

- The ampersand metacharacter is useful, but even more useful is the ability to define specific regions in regular expressions. These special regions can be used as reference in your replacement strings.
- By defining specific parts of a regular expression, you can then refer back to those parts with a special reference character. To do back references, you have to first define a region and then refer back to that region.
- To define a region, you insert backslashed parentheses around each region of interest. The first region that you surround with backslashes is then referenced by \1, the second region by \2, and so on.
- Assuming phone.txt has the following text
(555)555-1212
(555)555-1213
- Try the following command –
\$ cat phone.txt | sed 's/\((.*)\) \((.*-)\) \((.*\$\) /Area code: \1 Second: \2 Third: \3/'
Area code: (555) Second: 555- Third: 1212
Area code: (555) Second: 555- Third: 121

grep

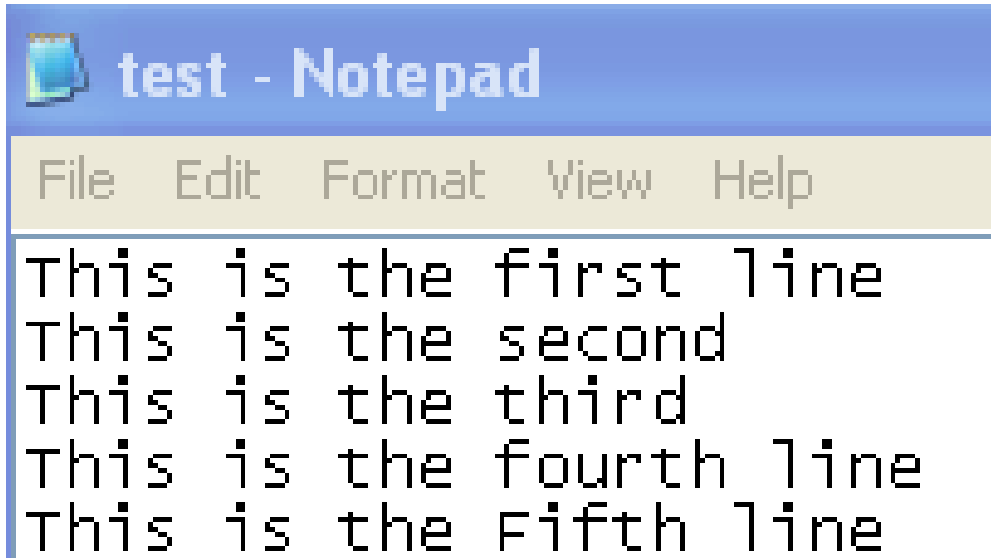
- grep is a command that searches through the input file for a specified pattern
- When grep finds a match to the pattern, it prints the entire line to standard output
- grep general structure:
 - grep options pattern input_file_names

Options (Matching Control)

- grep has a variety of options that can execute a wide range of operations once a match is found.
- '-i'
 - Ignores case.
- '-v'
 - Inverts the matching. When used, grep will print out lines that do not match the pattern
- '-e' pattern
 - Pattern is the pattern. This can be used to specify multiple patterns, or if the pattern starts with a '-'. A line only has to contain one of the patterns to be matched.

Examples using '-i', '-v', and '-e'

- The file below was used for these examples



```
test - Notepad
File Edit Format View Help
This is the first line
This is the second
This is the third
This is the fourth line
This is the Fifth line
```

```
Colin@masterson1 /cygdrive/c/Documents a
h_year/cs265
$ grep -i 'LINE' test.txt
This is the first line
This is the fourth line
This is the Fifth line

Colin@masterson1 /cygdrive/c/Documents a
h_year/cs265
$ grep -v 'line' test.txt
This is the second
This is the third

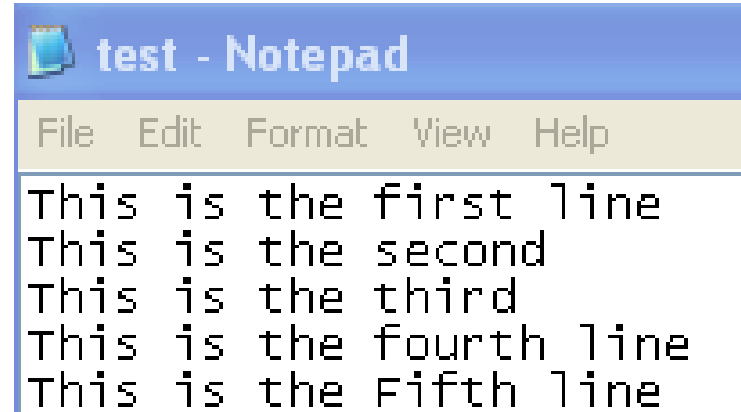
Colin@masterson1 /cygdrive/c/Documents a
h_year/cs265
$ grep -e 'fourth' -e 'first' test.txt
This is the first line
This is the fourth line
```

General Output Control

- `'-c'`
 - Suppress normal output and instead print out a count of matching lines for each input file
- `'-l'`
 - Suppress normal output and print the name of each file that contains at least one match
- `'-L'`
 - Suppress normal output. Print the name of each file that does not contain any matches
- Note: both the `'-l'` and `'-L'` options will stop searching a file once a match is found

Examples using '-c', '-l', and '-L'

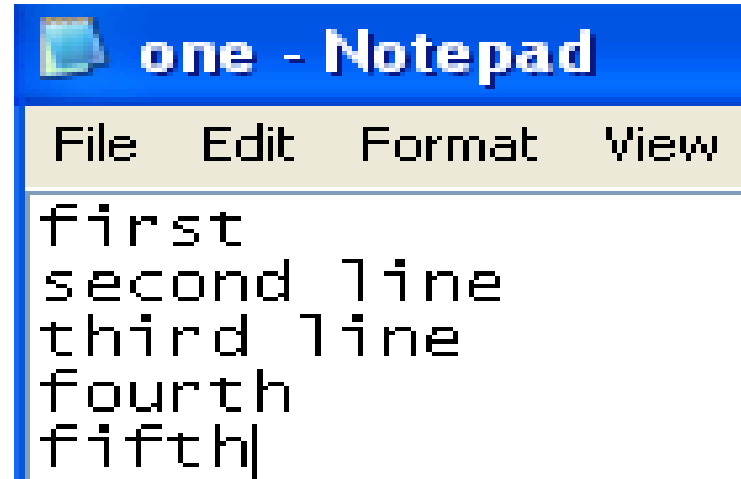
- Two files were used and specified below



test - Notepad

File Edit Format View Help

This is the first line
This is the second
This is the third
This is the fourth line
This is the Fifth line



one - Notepad

File Edit Format View

first
second line
third line
fourth
fifth

```
Colin@masterson1 /cygdrive/c/Document
h_year/cs265
$ grep -c 'line' test.txt one.txt
test.txt:3
one.txt:2
```

```
Colin@masterson1 /cygdrive/c/Document
h_year/cs265
$ grep -l 'This' test.txt one.txt
test.txt
```

```
Colin@masterson1 /cygdrive/c/Document
h_year/cs265
$ grep -L 'This' test.txt one.txt
one.txt
```

Output Line Prefix Control

- '-n'
 - Prefixes each line of output with the line number from the input file the match was found on
- '-H'
 - Prefix each line of output with the input file name that the match was found in
- '-T'
 - Makes sure that the actual line content (or whatever content comes after the '-T') lands on a tab stop

Examples using '-H', '-n', and '-T'

- Two files were used and specified below

test - Notepad

File Edit Format View Help

This is the first line
This is the second
This is the third
This is the fourth line
This is the Fifth line

one - Notepad

File Edit Format View

first
second line
third line
fourth
fifth|

```
Colin@masteron1 /cygdrive/c/Documents and Settings/Colin/My Documents/h_year/cs265
$ grep -HTnT 'line' test.txt one.txt
test.txt: 1 :This is the first line
test.txt: 4 :This is the fourth line
test.txt: 5 :This is the Fifth line
one.txt: 2 :second line
one.txt: 3 :third line
```

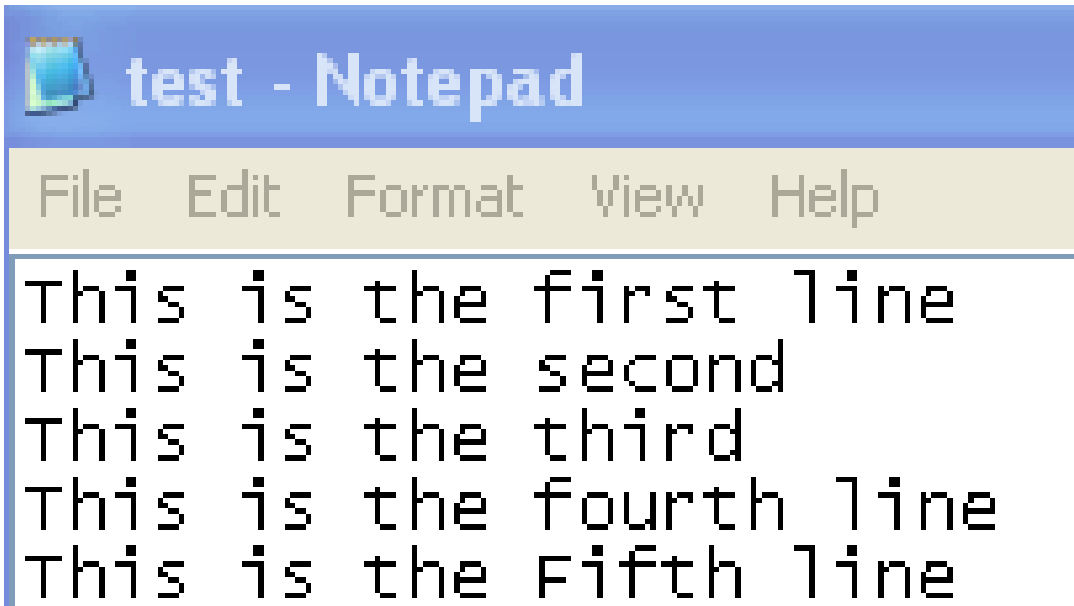
```
Colin@masteron1 /cygdrive/c/Documents and Settings/Colin/My Documents/h_year/cs265
$ grep -Hn 'line' test.txt one.txt
test.txt:1:This is the first line
test.txt:4:This is the fourth line
test.txt:5:This is the Fifth line
one.txt:2:second line
one.txt:3:third line
```

Context Line Control

- *'-A num'*
 - Print *num* lines of trailing context after matching lines
- *'-B num'*
 - Print *num* lines of leading context before matching lines
- *'-C num'* or *'-num'*
 - Print *num* lines of leading and trailing output context

Examples using '-A', '-B', and '-C'

- The file below was used for these examples



```
test - Notepad
File Edit Format View Help
This is the first line
This is the second
This is the third
This is the fourth line
This is the Fifth line
```

```
Colin@masterson1 /cygdrive/c
h_year/cs265
$ grep -A 1 'third' test.txt
This is the third
This is the fourth line

Colin@masterson1 /cygdrive/c
h_year/cs265
$ grep -B 1 'third' test.txt
This is the second
This is the third

Colin@masterson1 /cygdrive/c
h_year/cs265
$ grep -C 1 'third' test.txt
This is the second
This is the third
This is the fourth line

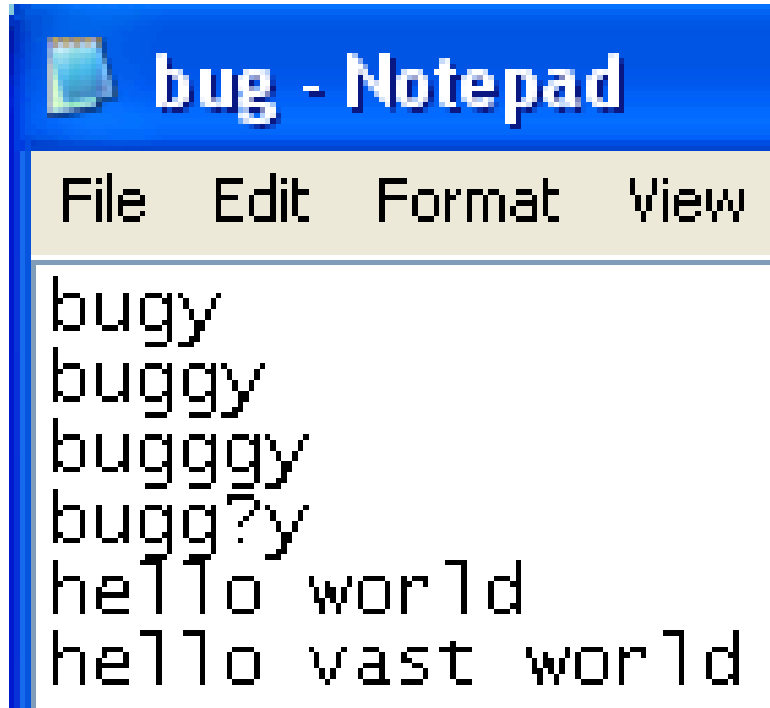
Colin@masterson1 /cygdrive/c
h_year/cs265
$ grep -l 'third' test.txt
This is the second
This is the third
This is the fourth line
```

Basic vs Extended Regular Expressions

- `'-G'`
 - Interpret pattern as basic regular expression (BRE). This is the default.
- `'-E'`
 - Interpret pattern as extended regular expression (ERE)
- When using basic regular expression some special characters (like `'?'` in the previous example) lose their special meaning and must have a `'\'`, the escape character, before them
- When using ERE, the escape character is unnecessary

BRE and ERE Difference

- Note that without the `\` in the BRE call (example 3), the `'?'` is seen as a normal character



```
bug - Notepad
File Edit Format View
bugy
buggy
buggyy
buggy?y
hello world
hello vast world
```

```
Colin@masterson1 /cygdrive/c
h_year/cs265
$ grep 'bugg\?y' bug.txt
bugy
buggy
```

```
Colin@masterson1 /cygdrive/c
h_year/cs265
$ grep -E 'bugg?y' bug.txt
bugy
buggy
```

```
Colin@masterson1 /cygdrive/c
h_year/cs265
$ grep 'bugg?y' bug.txt
buggy?y
```

Bracket Expressions

- A bracket expression is a list of characters enclosed by '[' and ']'. It matches any single character in the list
- However, if the first character in the list is '^', it matches any character not in the list
- A range can be done by using '-' in a bracket expression
 - [0-5] is the same as [012345]
- Some ranges are pre-defined in character classes
 - [:digit:] is the same as 0123456789
 - When using grep, the class name (including brackets) must be contained within another set of brackets

Bracket Expression Example

test2 - Notepad

File Edit Format

first line
2nd line
3rd
fourth line
5th line

```
Colin@masteron1 /cygdrive/c/Do  
h_year/cs265
```

```
$ grep '[r]' test2.txt
```

```
first line
```

```
3rd
```

```
fourth line
```

```
Colin@masteron1 /cygdrive/c/Do  
h_year/cs265
```

```
$ grep '[rd]' test2.txt
```

```
first line
```

```
2nd line
```

```
3rd
```

```
fourth line
```

```
Colin@masteron1 /cygdrive/c/Do  
h_year/cs265
```

```
$ grep '[:digit:]' test2.txt
```

```
2nd line
```

```
3rd
```

```
5th line
```

Matching Parts of Words

- Regular expressions have a concept of a “word” which is a little different than an English word.
 - A word is a pattern containing only letters, digits, and underscores ()
- Match beginning of a word with `<`
 - `<Fo` matches `Fo` if it appears at the beginning of a word
- Match the end of a word with `>`
 - `ox>` matches `ox` if it appears at the end of a word
- Whole words can be matched too: `<Fox>`

More Regular Expressions

- Matching the complement of a set by using the ^
 - [^aeiou] - matches any non-vowel
 - ^[^a-z]*\$ - matches any line containing no lower case letters
- Regular expression escapes
 - Use the \ (backslash) to “escape” the special meaning of wildcards
 - CA*Net
 - This is a full sentence\.
 - array\[3]
 - \[.*\]

Regular Expressions Recall

- A way to refer to the **most recent match**
- To remember portions of regular expressions
 - Surround them with **\(...\)**
 - Recall the remembered portion with **\n** where **n** is 1-9
 - Example: **'^\[a-z]\1'**
 - matches lines beginning with a pair of duplicate (identical) letters
 - Example: **'^.*\[a-z]*\1.*\1'**
 - matches lines containing at least three copies of something which consists of lowercase letters

Matching Specific Numbers of Repeats

- $X\{m,n\}$ matches m -- n repeats of the one character regular expression X
 - E.g. $[a-z]\{2,10\}$ matches all sequences of 2 to 10 lowercase letters
- $X\{m\}$ matches exactly m repeats of the one character regular expression X
 - E.g. $\#\{23\}$ matches 23 #s
- $X\{m,\}$ matches at least m repeats of the one character regular expression X
 - E.g. $^[aeiou]\{2,\}$ matches at least 2 vowels in a row at the beginning of a line
- $.\{1,\}$ matches more than 0 characters

Regular Expression Examples (1)

- How many lines in /usr/dict/words end in ing?

- `grep -c 'ing$' /usr/dict/words`

The -c option
says to count the
number of matches



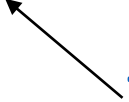
- How many lines in /usr/dict/words start with un and end with g?

- `grep -c '^un.*g$' /usr/dict/words`

- How many lines in /usr/dict/words begin with a vowel?

- `grep -ic '^[aeiou]' /usr/dict/words`

The -i option
says to ignore
case distinction



Regular Expression Examples (2)

- How many words in /usr/dict/words have triple letters in them?
 - `grep -ic '\(.\\)\1\1' /usr/dict/words`
- How many words in /usr/dict/words start and end with the same 3 letters?
 - `grep -c '^(...\\).*\1$' /usr/dict/words`
- How many words in /usr/dict/words contain runs of 4 consonants?
 - `grep -ic '[^aeiou]\{4\}' /usr/dict/words`

Regular Expression Examples (3)

- What are the 5 letter palindromes present in /usr/dict/words?
 - `grep -ic '^\(.\)\(.\).\2\1$' /usr/dict/words`
- How many words of the words in /usr/dict/words with y as their only vowel
 - `grep '^[^aAeEiloOuU]*$' /usr/dict/words | grep -ci 'y'`
- How many words in /usr/dict/words do not start and end with the same 3 letters?
 - `grep -ivc '^\(...\)\.*\1$' /usr/dict/words`

Awk

Programmable Filters

Why is it called AWK?



Aho



Weinberger



Kernighan

Awk Introduction

- **awk**'s purpose: A general purpose programmable filter that handles text (strings) as easily as numbers
 - This makes **awk** one of the most powerful of the Unix utilities
- **awk** processes *fields* while **sed** only processes lines
- **nawk** (new **awk**) is the new standard for **awk**
 - Designed to facilitate large **awk** programs
 - **gawk** is a free **nawk** clone from GNU
- **awk** gets its input from
 - files
 - redirection and pipes
 - directly from standard input

Structure of an AWK Program

- An **awk** program consists of:
 - An optional BEGIN segment
 - For processing to execute prior to reading input
 - pattern - action pairs
 - Processing for input data
 - For each pattern matched, the corresponding action is taken
 - An optional END segment
 - Processing after end of input data

```
BEGIN {action}  
  
pattern {action}  
  
pattern {action}  
  
.  
  
.  
  
.  
  
pattern { action}  
  
END {action}
```

Running an AWK Program

- There are several ways to run an Awk program
 - ***awk 'program' input_file(s)***
 - program and input files are provided as command-line arguments
 - ***awk 'program'***
 - program is a command-line argument; input is taken from standard input (yes, awk is a filter!)
 - ***awk -f program_file input_files***
 - program is read from a file

Pattern-Action Structure

- Every program statement has to have a *pattern* **or** an *action* **or** both
- Default *pattern* is to match all lines
- Default *action* is to print current record
- Patterns are simply listed; actions are enclosed in { }
- **awk** scans a sequence of input *lines*, or *records*, one by one, searching for lines that match the pattern
 - Meaning of match depends on the pattern

Patterns

- Selector that determines whether *action* is to be executed
- *pattern* can be:
 - the special token **BEGIN** or **END**
 - regular expressions (enclosed with `//`)
 - arithmetic relation operators
 - string-valued expressions
 - arbitrary combination of the above
 - `/NYU/` matches if the string “NYU” is in the record
 - `x > 0` matches if the condition is true
 - `/NYU/ && (name == "UNIX Tools")`

BEGIN and END patterns

- **BEGIN** and **END** provide a way to gain control before and after processing, for initialization and wrap-up.
 - **BEGIN**: actions are performed before the first input line is read.
 - **END**: actions are done after the last input line has been processed.

Actions

- *action* may include a list of one or more C like statements, as well as arithmetic and string expressions and assignments and multiple output streams.
- *action* is performed on every line that matches *pattern*.
 - If *pattern* is not provided, *action* is performed on every input line
 - If *action* is not provided, all matching lines are sent to standard output.
- Since *patterns* and *actions* are optional, *actions* must be enclosed in braces to distinguish them from *pattern*.

An Example

- `ls | awk ' BEGIN { print "List of html files:" } /\.html$/ { print } END { print "There you go!" } '`

```
List of html files:
index.html
as1.html
as2.html
There you go!
```

An Example

- `awk 'BEGIN { print "Hello header" } { print $2 } END { print "Ending....."}' file1.txt`
- `awk -f example1.awk file1.txt`
- `cat file1.txt | awk -f example1.awk`

An Example: Changing field separator

- `awk 'BEGIN{FS=","} { print $1}' file2.txt`
- `cat /etc/passwd | awk 'BEGIN{FS=":"} { print $1 }'`
- `cat file1.txt | awk '{print $2, $3,$4}'`
- `cat file2.txt | awk 'BEGIN{FS=","; OFS="-"}{print $1,$2}'`

An Example: Changing record separator

- `awk 'BEGIN{RS=":";FS=","} { print $1 }' file3.txt`
- `awk 'BEGIN{RS=":";FS=","; ORS="|"} { print $1, $2 }' file3.txt`

An Example: Regular expression

- `cat file4.txt | awk '/fun/ { print $1 } '`
- `cat file4.txt | awk '/f.n/ { print $1, $2,$3 } '`
- `cat file4.txt | awk '$0 ~ /f.n/ { print $1, $2,$3 } '`
- `cat file4.txt | awk '$0 !~ /f.n/ { print $1, $2,$3 } '`
- `cat file4.txt | awk '$2 ~ /f.n/ { print $1, $2,$3 } '`

Examples :Arithmetic operations

- `awk '{ print $2+$2+$3}' file5.txt`
- `awk '{ sum=$2+$3+$4; print "Sum is:", sum }' file5.txt`
- `awk '{ avg=($2+$3+$4)/3; print "Avg is:", avg }' file5.txt`

Examples :if else

- `awk -f example2.awk file6.txt`

```
BEGIN{
    FS=","
    infant=0
    adult=0
    senior=0
}

if ($NF < 18)
{
    infant++
}
else if ($NF >18 && $NF < 61)
{
    adult++
}
else
{
    senior++
}

END{
    print "Number of infants:", infant
    print "Number of adults:", adult
    print "Number of senior:", senior
}
```